UNIVERSIDAD NACIONAL DE COLOMBIA
FACULTAD DE INGENIERÍA
Ingeniería de Software II

Principios SOLID

Autor:
Juan José Martin Vargas - jmartinv@unal.edu.co

Docente de la asignatura:
Sergio Enrique Vargas Pedraza

BOGOTÁ D.C
Febrero del 2026

Principios SOLID

Los principios SOLID son un conjunto de cinco principios de diseño de programación que buscan hacer que los diseños de código sean más legibles y limpios.

Acrónimo	Principio	Significado
S	Single Responsibility Principle (SRP)	Una clase debe tener solo una razón para cambiar, es decir, debe tener una única responsabilidad.
O	Open/Closed Principle (OCP)	Las entidades de software (clases, módulos, funciones, deben estar abiertas para la extensión, pero cerradas para la modificación.
L	Liskov Substitution Principle (LSP)	Los objetos en un programa deben ser reemplazables por instancias de sus subtipos sin alterar la corrección de ese programa.
I	Interface Segregation Principle (ISP)	Los clientes no deben ser forzados a depender de interfaces que no utilizan. Es mejor tener muchas interfaces específicas que una sola interfaz grande.
D	Dependency Inversion Principle (DIP)	Los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones.

Principios SOLID son cruciales para el desarrollo de software robusto y de calidad.

- SOLID asegura que:
  - Cada clase tenga una sola responsabilidad (SRP).
  - Se pueda añadir funcionalidad sin modificar código existente (OCP).
  - Una subclase sustituya a su clase padre sin errores (LSP).
  - Las clases no dependan de interfaces que no usan (ISP).
  - Los detalles técnicos dependan de abstracciones/interfaces (DIP).
- Seguir SOLID es fundamental para una arquitectura de software de primera, garantizando sistemas:
  - Que crecen sin problemas.
  - Fáciles de mantener.
  - Adaptables a cambios futuros.

Ejemplo práctico de Single Responsibility Principle (SRP):

```javascript
#Ejemplpo incorrecto

class Usuario {
  constructor(nombre, email) {
    this.nombre = nombre
    this.email = email
  }

  saveToDB() {
    console.log("Guardando usuario en la base de datos")
  }

  sendEmail() {
    console.log("Enviando email al usuario")
  }

  generateReport() {
    console.log("Generando reporte del usuario")
  }
}
```

Esta clase:

- Maneja datos del usuario
- Guarda en base de datos
- Envía correos electrónicos
- Genera reportes

```javascript
#Ejemplo correcto

class Usuario {
  constructor(nombre, email) {
    this.nombre = nombre
    this.email = email
  }
}

class RepositorioUsuario{
  guardar(usuario) {
    console.log("Guardando usuario en la base de datos")
  }
}

class ServicioEmail{
  enviar(usuario) {
    console.log("Enviando email a " + usuario.email)
  }
}

class UserReport {
  generar(usuario) {
    console.log("Generando reporte de " + usuario.nombre);
  }
}
```

Ahora con el principio SOLID de Single Responsibility Principle (SRP):

- Si cambia la base de datos se debe modificar `RepositorioUsuario`
- Si cambia el sistema de correo, basta con modificar `ServicioEmail`
- Cada clase tiene una sola razón para cambiar.